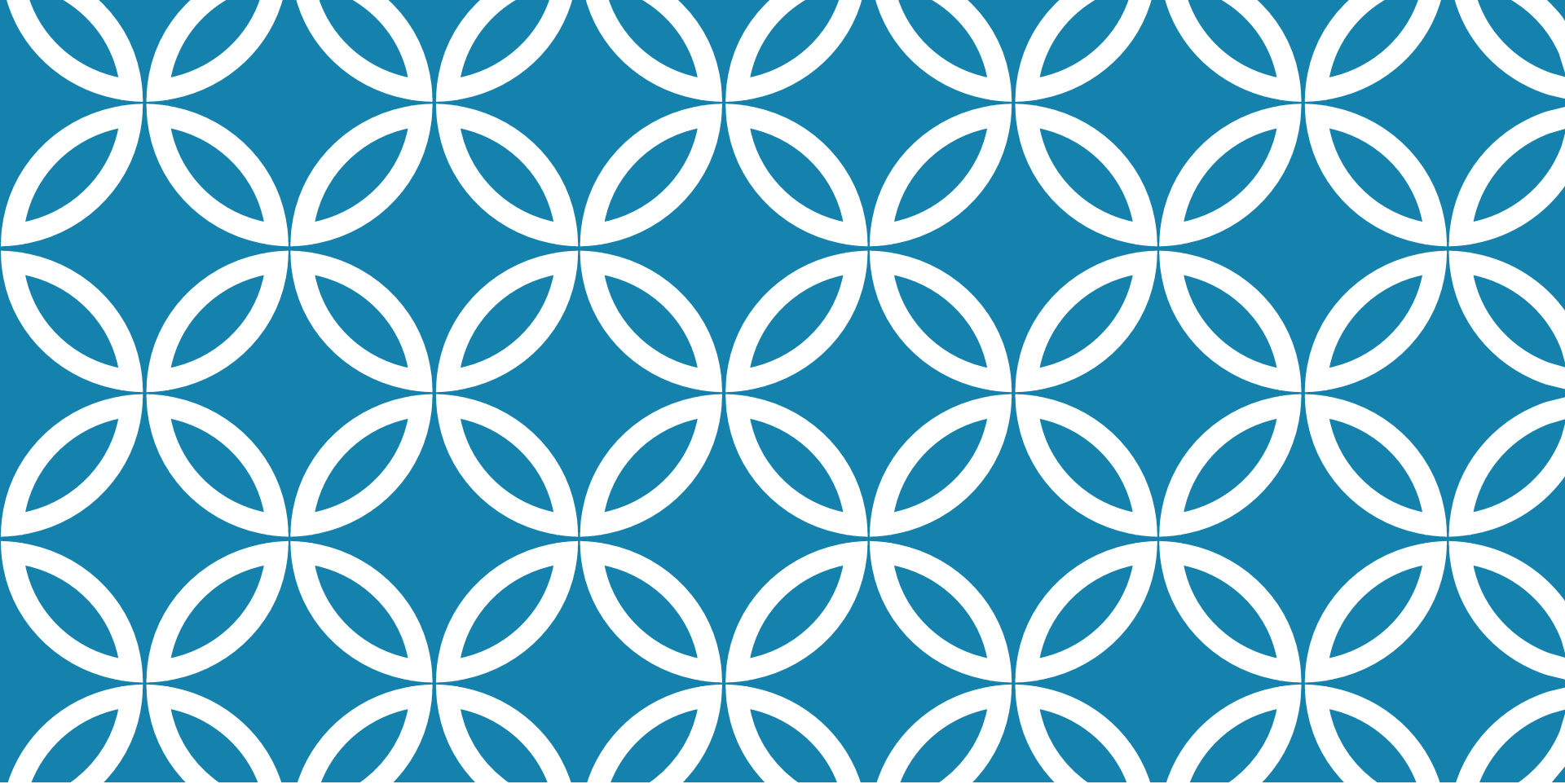




ORCHESTRATION-DRIVEN AND MICROSERVICES ARCHITECTURES

Lecture 7

LAST TIME

Architectural Styles

- Event-driven
 - Broker
 - Mediator

CONTENT

Architectural Styles

- Orchestration-driven service-oriented
- Microservices

REFERENCES

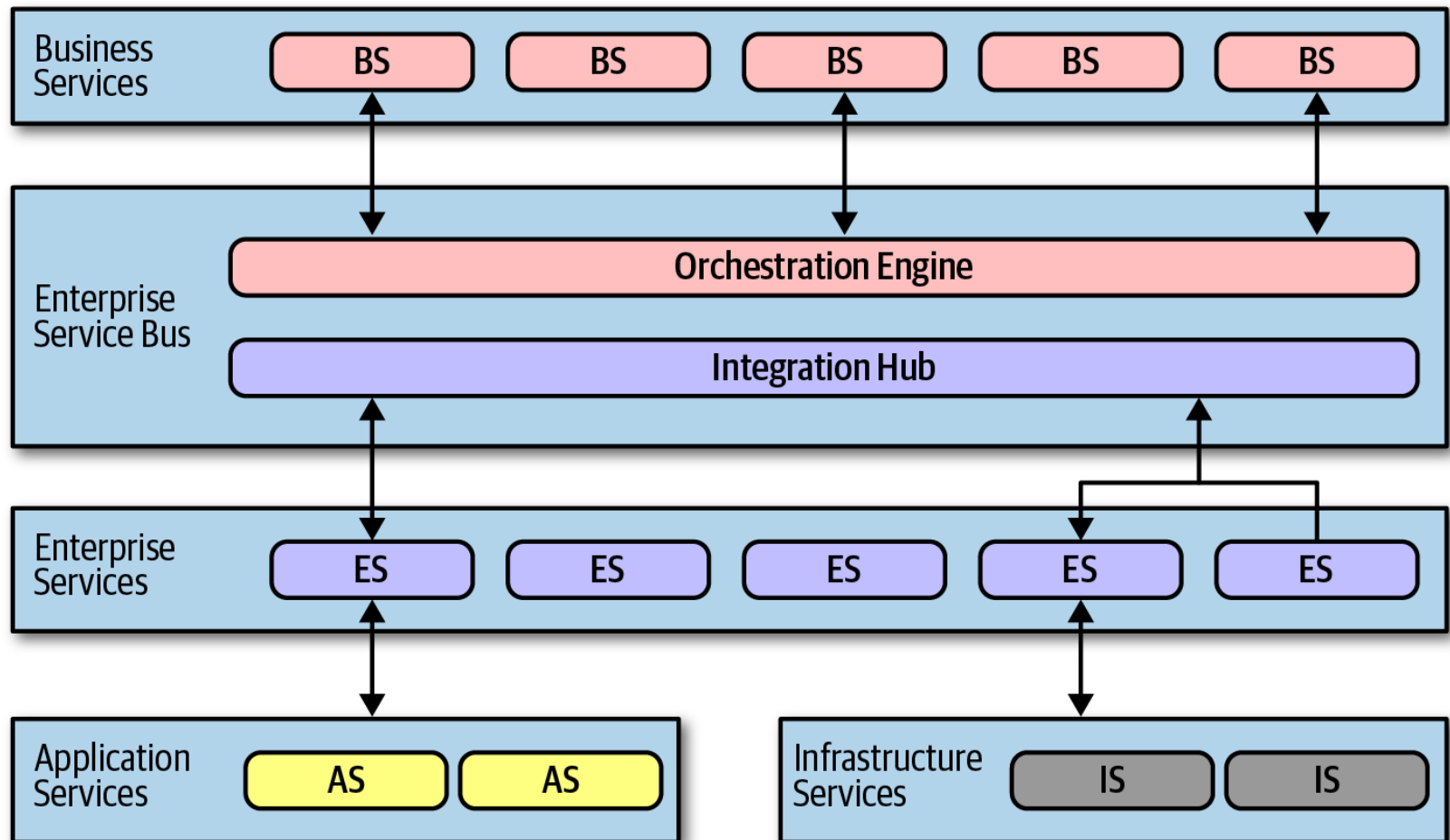
Fundamentals of Software Architecture, Mark Richards,
Neal Ford, 2020 - Chapters 16, 17, 18

MORE ON SERVICES

- Historical context
 - Late 1990s: merging companies => enterprises
 - Scarce, expensive computing resources
 - Emerging distributed computing
 - Scalability and flexibility is needed
 - Operating systems were expensive and licensed per machine
 - Commercial database servers came with complex licensing schemes
- ⇒ Architects tried to reuse as much as possible!
- ⇒ Orchestration driven architectures (good motivation, bad consequences!)

TOPOLOGY

- **technical partitioning**



TAXONOMY

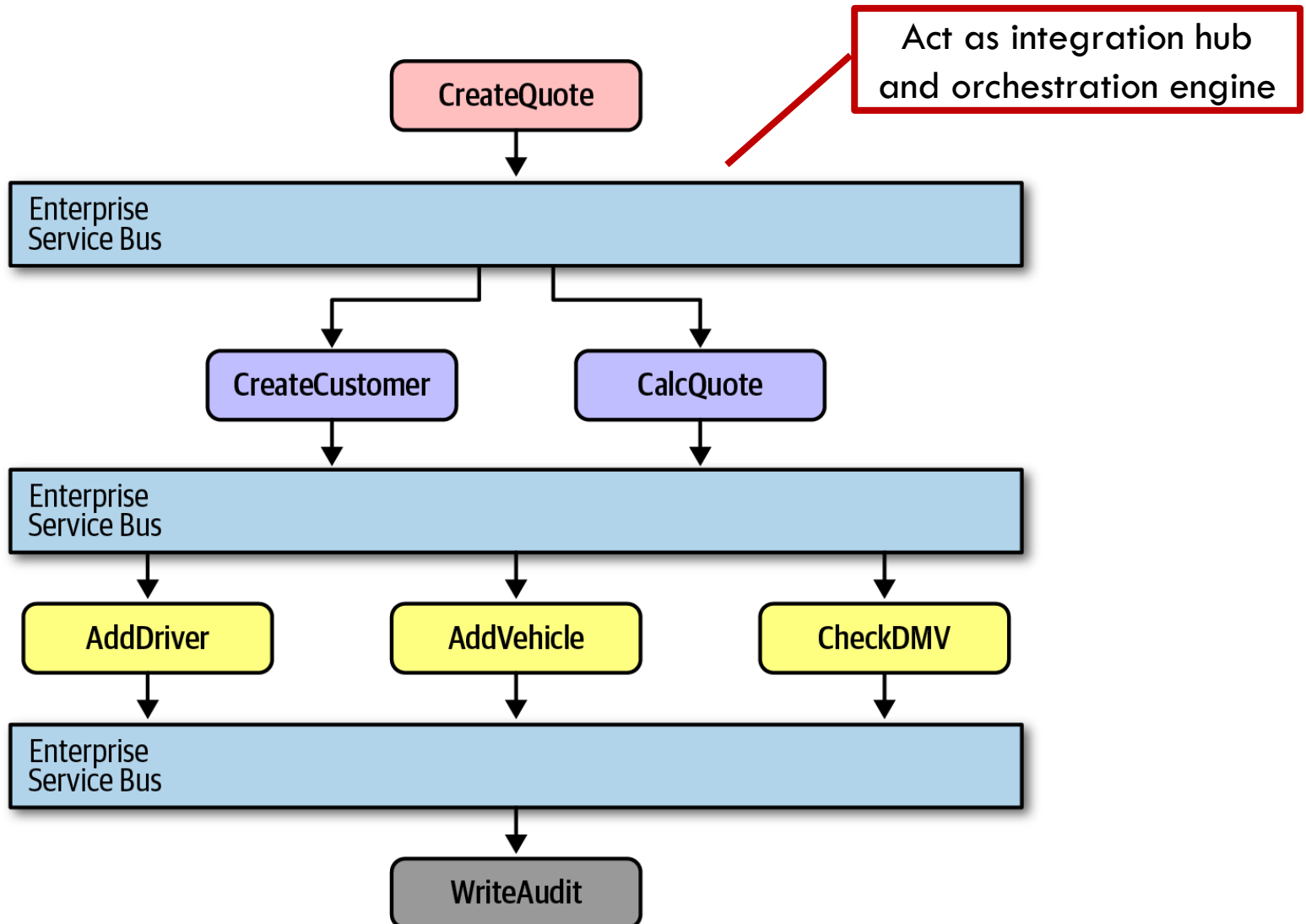
- driving philosophy is centered around **enterprise-level reuse**
- **Business services:**
 - Entry point of the architecture
 - Usually representing domain behavior (ex. ExecuteTrade, PlaceOrder)
 - Usually defined by business users (no code, just input, output)
- **Enterprise Services**
 - fine-grained, shared implementations (ex. CreateCustomer, CalculateQuote)
 - Building blocks that make up coarse-grained business services
 - Tied together by orchestration engine

Reality is dynamic => very difficult to build fine-grained enterprise services at just the correct level of granularity!

TAXONOMY

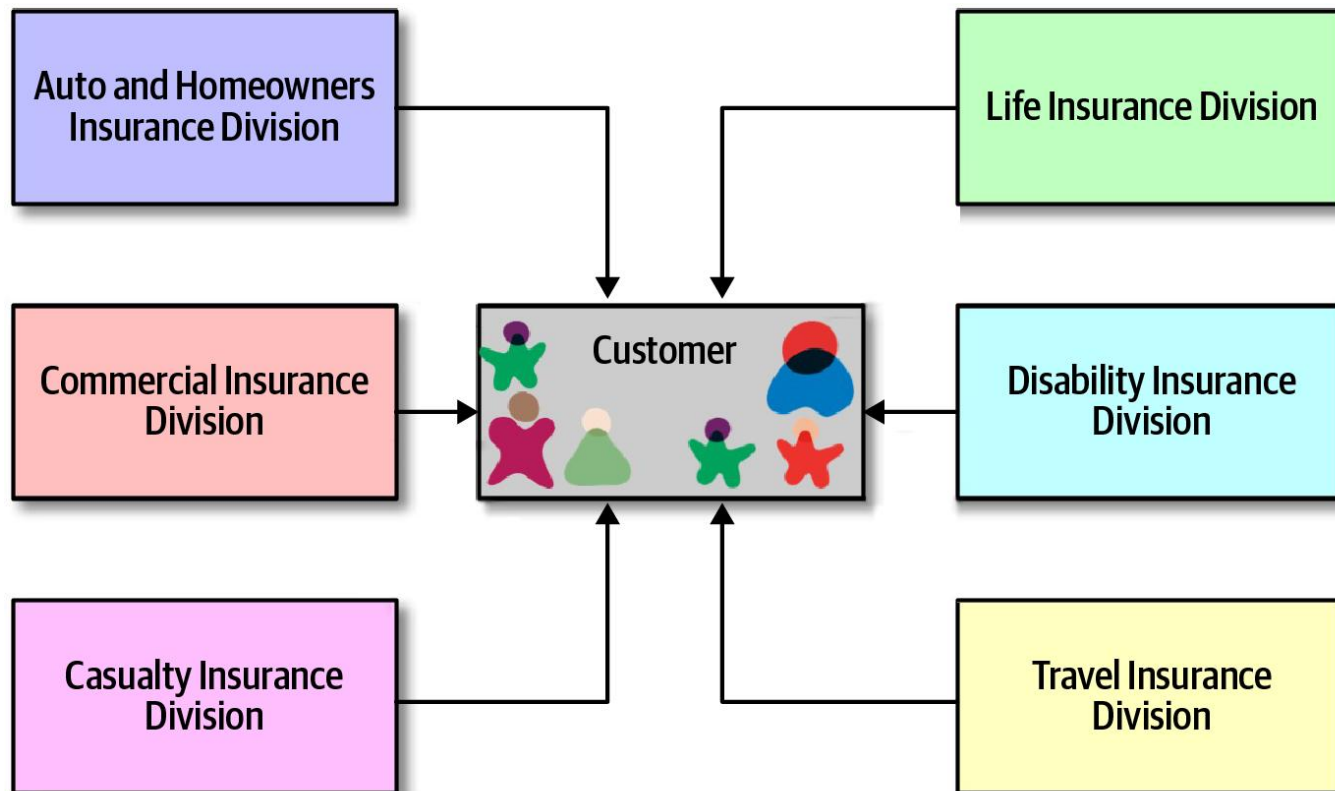
- **Application** services:
 - one-off, single-implementation services
 - Not all services in the architecture require the same level of granularity or reuse as the enterprise services
 - typically owned by a single application team
- **Infrastructure** services
 - supply the operational concerns (ex. monitoring, logging, authentication, and authorization)
 - tend to be concrete implementations,
 - owned by a shared infrastructure team
- **Orchestration** Engine
 - keeps all together by using transactional coordination and message transformation
 - typically tied to a single relational database (or a small number of databases)
 - defines the relationship between the business and enterprise services
 - acts as an integration hub

MESSAGE FLOW



REUSE

- Objective: reuse at service level => gradually build business behavior that can be incrementally reused over time



WHAT CAN GO WRONG?

- **Huge amount of coupling** between components

=> Coordinated deployments, holistic testing

- The reused service contains ALL the information (not all using components need it)

=> Unnecessary complexity

- Dozens of services in several tiers + one single database schema => Difficult to maintain

ARCHITECTURE CHARACTERISTICS RATING

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★ ★ ★
Evolutionary	★
Fault tolerance	★ ★ ★
Modularity	★ ★ ★
Overall cost	★
Performance	★ ★
Reliability	★ ★
Scalability	★ ★ ★ ★
Simplicity	★
Testability	★

WHAT ARE SERVICES? DESIGN PRINCIPLES

Services are reusable

- Business functionalities exposed as services are designed with the intention of reuse whenever and where they are required

Services share a formal contract

- Services interact with each through a formal contract which is shared to exchange information and terms of usage

Services are loosely coupled

- Services are designed as loosely coupled entities able to interact while maintaining their state of loose coupling.

Services abstract underlying logic

- The business logic underpinning a service is kept hidden from the outside world. Only the service description and formal contract are visible for the potential consumers of a service

SERVICE DESIGN PRINCIPLES 2

Services are composable

- Services may be composed of other services. Hence, a service's logic should be represented at different levels of granularity and promotes reusability and the creation of abstraction layers.

Services are autonomous

- A service should be independent of any other service

Services are stateless

- A service shouldn't be required to maintain state information rather it should be designed to maximize statelessness

Services are discoverable

- A service should be discoverable through its description, which can be understood by humans and service users.

SERVICE DESIGN PRINCIPLES 3

Services have a network-addressable interface

- A service should be invoked from the same computer or remotely – through a local interface or Internet

Services are location transparent

- A service should be discoverable without the knowledge of its real location. A requestor can dynamically discover the location of a service looking up a registry.

The core principles are **autonomy, loose coupling, abstraction, formal contract**

WHAT IS A WEB SERVICE? (W3C DEFINITION)

A Web service:

- is a software system designed to support **interoperable machine-to-machine interaction** over a network.
- has an **interface** described in a machine-processable format (ex. WSDL for SOAP-based services).

Other systems interact with the Web service in a manner prescribed by its description using SOAP-messages, typically conveyed using HTTP with an XML serialization in conjunction with other Web-related standards.

WHAT IS A WEB SERVICE: A SIMPLER DEFINITION

A Web Service is a standards-based way for an application to call a function over a network and to do it without having to know:

- the **location** where the function will be executed,
- the **platform** where the function will be run,
- the **programming language** it is written in, or even
- **who** built it.

WEB SERVICES - SOAP BASED

Discovery

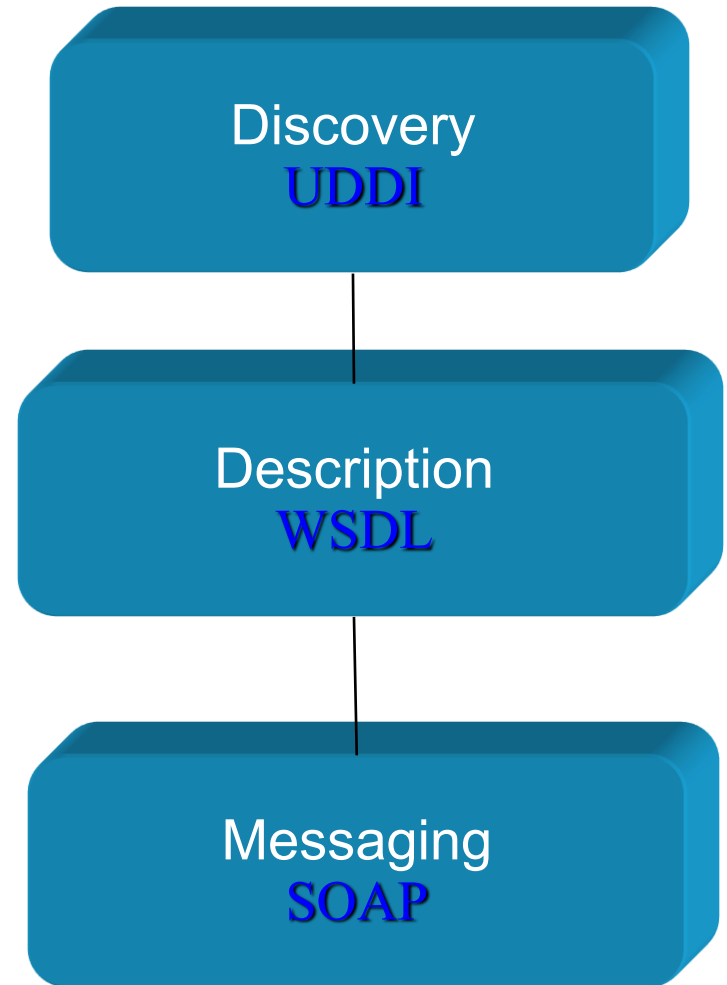
- Where is the service?

Description

- What service does it offer?
- How do I use it?

Messaging

- Let's communicate!



STANDARD IS KEY

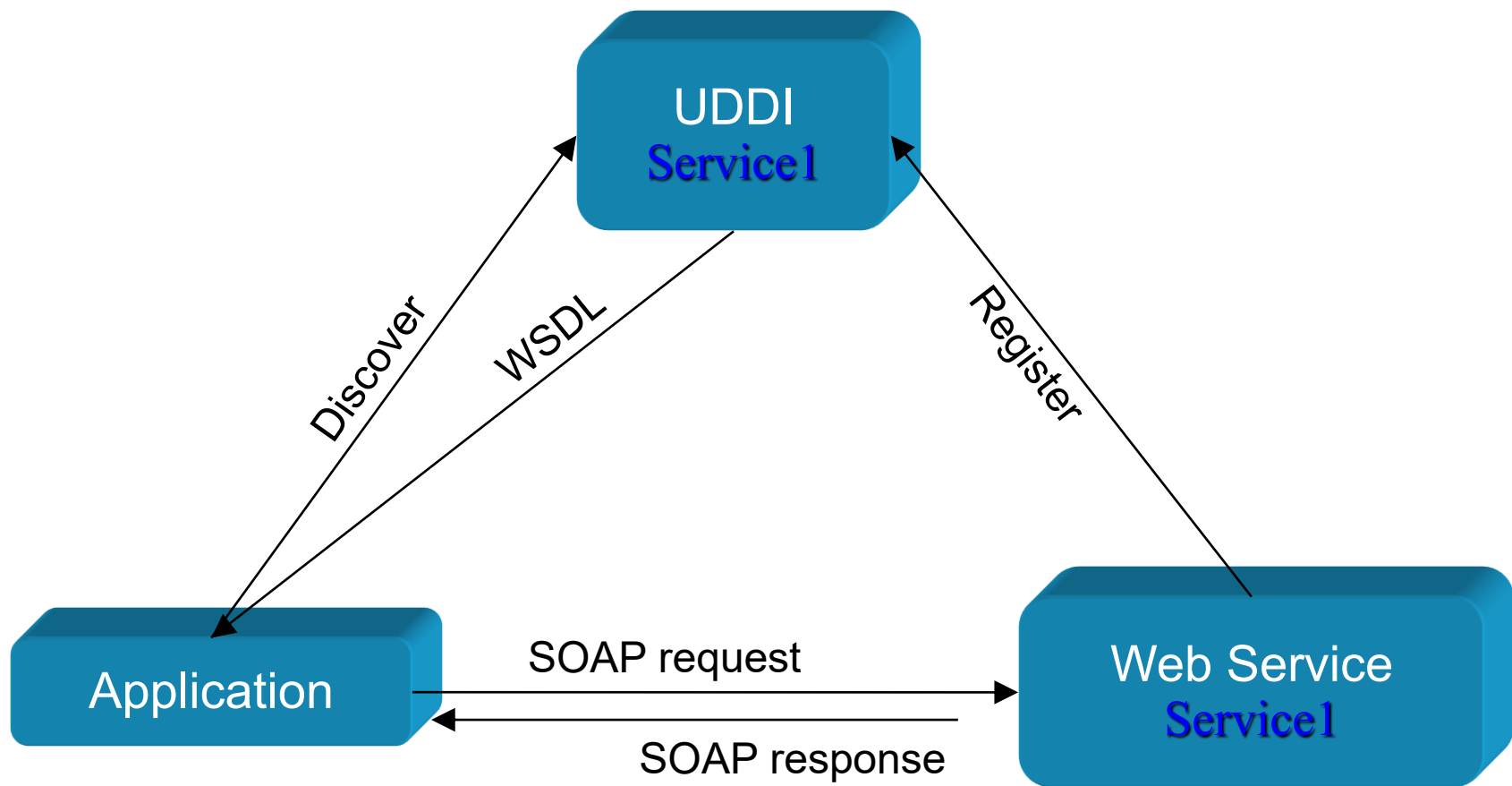
WSDL is used to describe the **function(s)** that an application will be calling documenting in a standard way its entry points, parameters and output

XML is used to carry the values of **parameters** and the **outputs** of the function

SOAP is used as the **messaging protocol** that carries content (XML) over a network transport (typically HTTP)

HTTP is used as the **network transport layer**

INTERACTION



WSDL: WEB SERVICE DESCRIPTION LANGUAGE

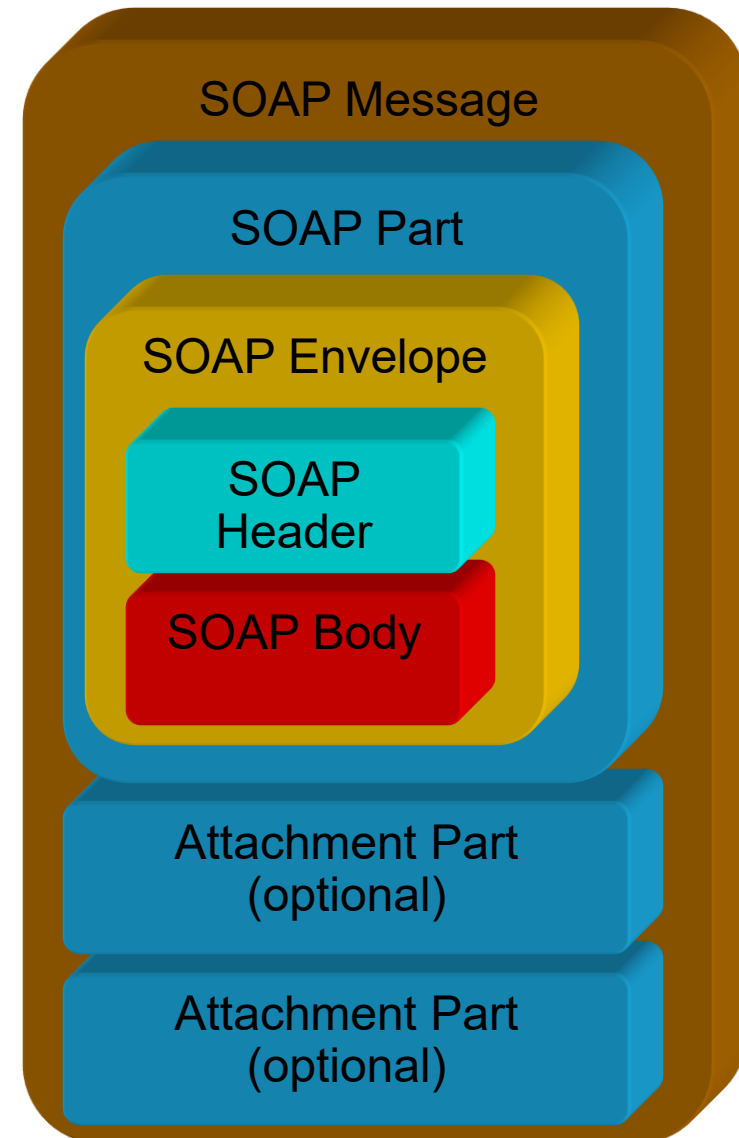
- WSDL (Web Services Description Language) is a public description of the interfaces offered by a web service.
- Expressed in XML
- Describes services as a set of endpoints
 - Document-oriented
 - Procedure-oriented

XML grammar

- `<definitions>`: root WSDL element
- `<types>`: data types transmitted (starts with XML Schema specifications)
- `<message>`: messages transmitted
- `<portType>`: functions supported
- `<binding>`: specifics of transmissions
- `<service>`: how to access it

SOAP: A DESCRIPTION

- Industry standard message format for sending and receiving data between a web services consumer and a web service provider
- SOAP messages are XML documents which have an envelope and:
 - Header (optional): contains information about the message such as date/time it was sent or security information
 - Body: contains the message itself
- SOAP stands for Simple Object Access Protocol



GIVE IT A REST

REST: **RE**presentation **S**tate **T**ransfer

Rest-ful: Follows the REST principles

Not strictly for web services

Term used loosely as a method of sending information over HTTP without using a messaging envelope

REPRESENTATIONAL STATE TRANSFER (REST)

- Idea: *Self-contained* requests specify what *resource* to operate on and what to do to it [Roy Fielding's PhD thesis, 2000]
- A service (in the SOA sense) whose follows the REST principles (next slide) is a RESTful service
- Ideally, RESTful URIs name the operations

REST PRINCIPLES

[RP1] The key abstraction of information is a **resource**, named by an URL. Any information that can be named can be a resource.

[RP2] The **representation** of a resource is a sequence of bytes, plus representation metadata to describe those bytes.

[RP3] All interactions are context-free: each interaction contains all of the information necessary to understand the request, independent of any requests that may have preceded it (**Stateless**).

REST PRINCIPLES (CONT'D)

[RP4] Components perform only a small set of well-defined methods on a resource producing a representation to capture the current or intended state of that resource and transfer that representation between components. These methods are global to the specific architectural instantiation of REST; for instance, all resources exposed via HTTP are expected to support each operation identically (**Uniform interface**).

[RP5] Idempotent operations and representation metadata are encouraged in support of **caching** and representation reuse.

[RP6] The presence of intermediaries is promoted. Filtering or redirection intermediaries may also use both the metadata and the representations within requests or responses to augment, restrict, or modify requests and responses in a manner that is transparent to both the user agent and the origin server (**Links between resources**).

RESOURCES

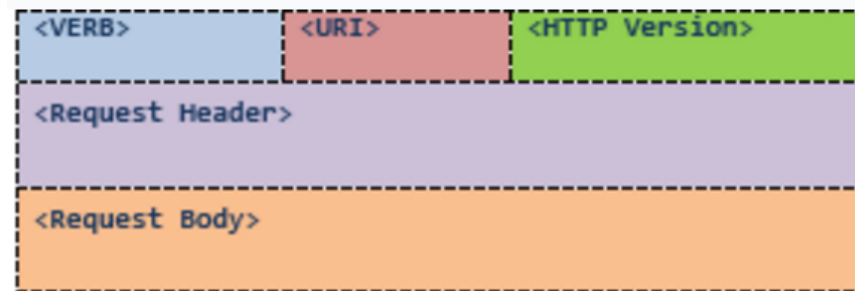
XML representation

```
<Person>
  <ID>1</ID>
  <Name>M Dinso</Name>
  <Email>m.dinso@gmail.com</Email>
  <Country>Romania</Country>
</Person>
```

JSON representation

```
{
  "ID": "1",
  "Name": "M Dinso",
  "Email":
    "m.dinso@gmail.com",
  "Country": "Romania"
}
```

MESSAGES — HTTP REQUEST



<VERB> is one of the HTTP methods like GET, PUT, POST, DELETE, OPTIONS, etc

<URI> is the URI of the resource on which the operation is going to be performed

<HTTP Version> is the version of HTTP

<Request Header> contains the metadata as a collection of key-value pairs of headers and their values. Ex. client type, the formats client supports, format type of the message body, cache settings for the response, etc.

<Request Body> is the actual message content. In a RESTful service, that's where the representations of resources sit in a message.

POST REQUEST EXAMPLE

POST http://MyService/Person/

Host: MyService

Content-Type: text/xml; charset=utf-8

Content-Length: 123

<?xml version="1.0" encoding="utf-8"?>

<Person>

<ID>1</ID>

<Name>M Dinso</Name>

<Email>m.dinso@gmail.com</Email>

<Country>Romania</Country>

</Person>

HTTP VERBS — UNIFORM INTERFACE

GET

- Retrieves a resource
- Guaranteed not to cause side-effect (SAFE)
- Cacheable

POST

- Creates a new resource
- Unsafe, effect of this verb isn't defined by HTTP

PUT

- Updates an existing resource
- Used for resource creation when client knows URI
- Can call N times, same thing will always happen (idempotent)

DELETE

- Removes a resource
- Can call N times, same thing will always happen (idempotent)

Taken from <https://www.oreilly.com/library/view/restful-net/9780596155025/ch04.html>

STATELESSNESS

- Does not maintain the application state for any client.
- A request cannot be dependent on a past request and a service treats each request independently.

Example:

Request1: GET http://MyService/Persons/1 HTTP/1.1

Request2: GET http://MyService/Persons/2 HTTP/1.1

SOAP VS. REST

Difference	SOAP	REST
Style	Protocol	Architectural Style
Function	Function-driven: transfer structured information	Data-driven: access a resource for data
Data format	Only uses XML	Permits several data formats: HTML, XML, plain text, JSON
Security	Supports WS-Security and SSL	Supports SSL and HTTPS
Bandwidth	Requires more resources and bandwidth	Requires fewer resources and is lightweight
Data cache	Cannot be cached	Can be cached
Payload handling	Has a strict communication contract and needs knowledge of everything before any interaction	Needs no knowledge of the API
Transactional support	Has built-in ACID compliance	Lacks ACID compliance

WHEN TO USE REST?

- **Limited resources and bandwidth**
- **Statelessness** – If there is no need to maintain a state of information from one request to another
- **Caching** – If there is a need to cache a lot of requests
- **Ease of coding**
- **Challenges**
 - **Lack of Security** – REST does not impose any sort of security like SOAP. This is why REST is very appropriate for public available URL's, but when it comes down to confidential data being passed between the client and the server, REST is the worst mechanism to be used for web services.
 - **Lack of state** – Most web applications require a stateful mechanism. For example, if you had a purchasing site which had the mechanism of having a shopping cart, it is required to know the number of items in the shopping cart before the actual purchase is made

WHEN TO USE SOAP?

- **Asynchronous processing and subsequent invocation**
- **A Formal means of communication**
- **Stateful operations**
- **Challenges**
 - **WSDL file:** the tight contract between the client and the server and one change could cause a large impact, on the whole, client applications
 - **Document size** – Large messages can be a problem for limited bandwidth

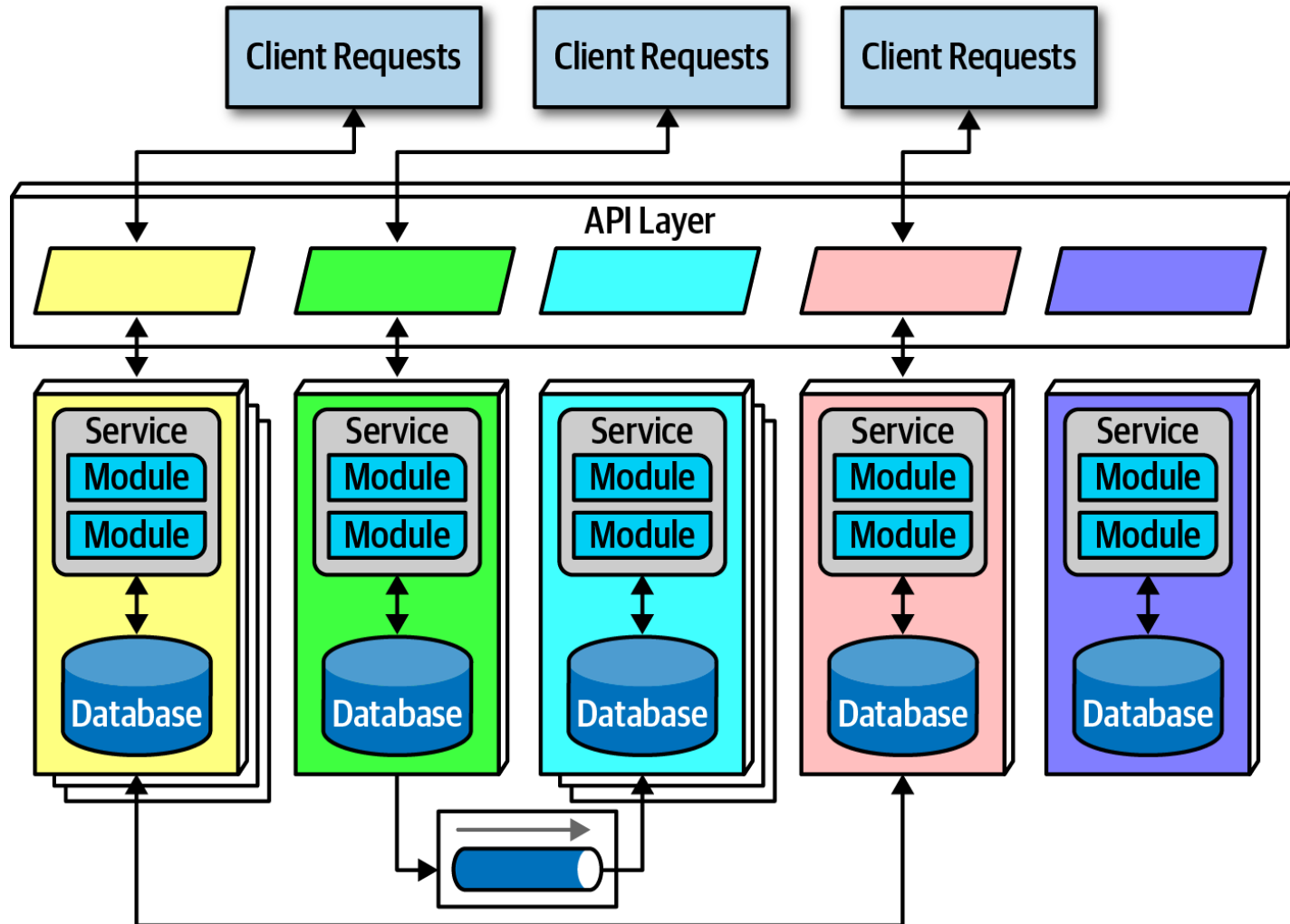
SOAP Services are like envelopes while REST services are like postcards

SOAP Services cannot use REST while REST can use SOAP.

MICROSERVICES ARCHITECTURE

- named early in its usage and popularized by a famous blog entry by Martin Fowler and James Lewis entitled “Microservices,” published in March 2014
- inspired by the ideas in **domain-driven design** (DDD) (i.e. bounded context)
- philosophy: **favor duplication over reuse => high decoupling**

MICROSERVICES TOPOLOGY



DISTRIBUTION

Benefits

- Each service runs in its own process (virtual machines and containers)
- Provides isolation
- Due to freely available open source operating systems and automated machine provisioning => more **affordable and practical**

Limitations

- Latency of network calls + security verification at every endpoint
=> high **latency**
- Transaction management => granularity of services to integrate transactions **within** service boundaries

GRANULARITY

- Microservices **DO NOT HAVE TO BE SMALL!**
- A service should contain everything necessary to operate within the application, including classes, other subcomponents, and database schemas
- The purpose is to capture a domain or workflow => **extremely functionally cohesive**
- Good practice to include entities that are involved in a transaction
- Good practice to include services that communicate heavily => avoid communication overhead

DATA ISOLATION

- Do not model services to resemble single entities in a database!
- Enable distribution of data across the architecture => no more single source of truth

⇒ Identify one domain as the source of truth and coordinate with it

OR

⇒ Use database replication to distribute information

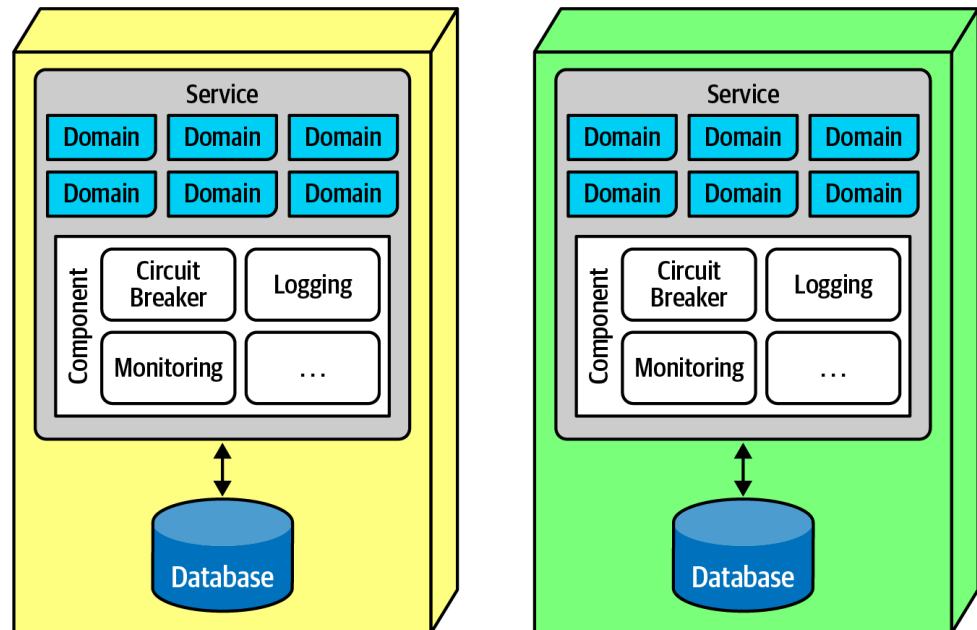
OPERATIONAL REUSE

- All interesting logic in this architecture should occur inside a bounded context

⇒ Orchestration or logic in a mediator is a violation!

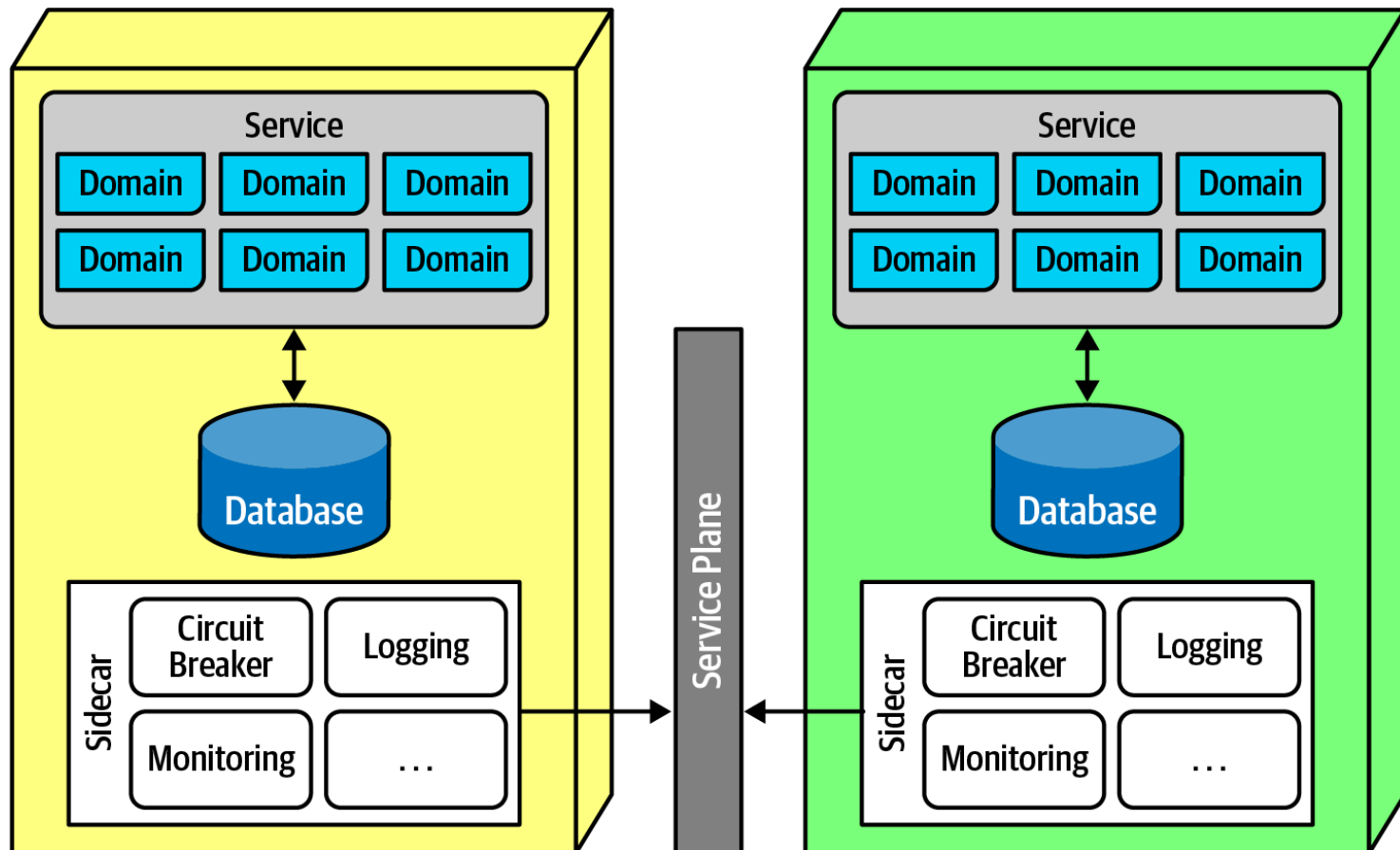
- What if several services have some similar elements?

⇒ Sidecar pattern



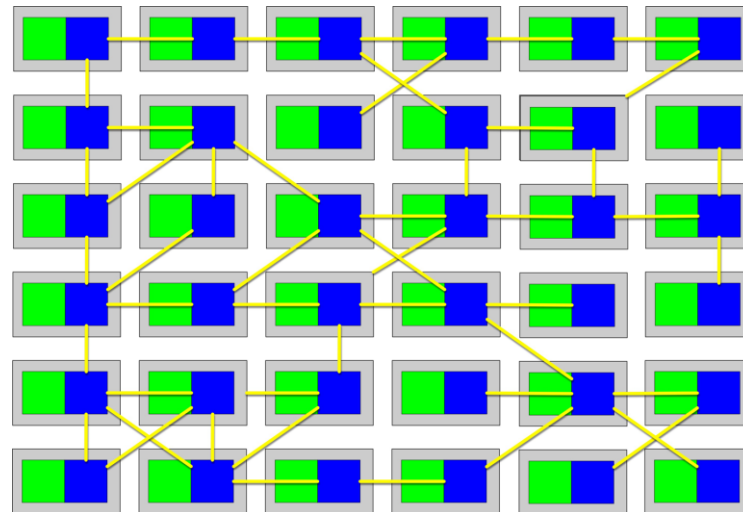
SERVICE MESH

- Each service includes a common sidecar => build a **service mesh** to form a consistent operational interface across all microservices



SERVICE CONSOLE

- Each sidecar wires into the service plane, which forms the consistent interface to each service => service console
- How is each service found?
 - ⇒ Service discovery tool
 - Monitors number and frequency of requests
 - Spins up new instances of services to handle scale or elasticity
 - Hosted by API Layer



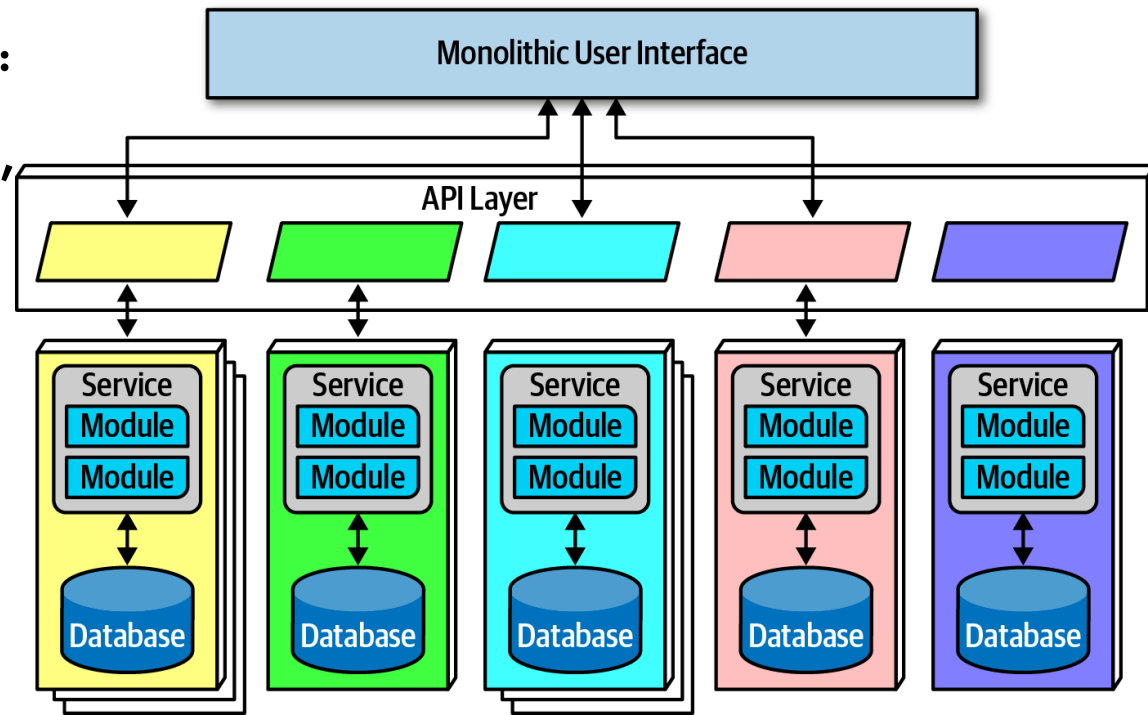
FRONTENDS

- Decoupling is not only about backend
- Original vision for microservices included the user interface as part of the bounded context

⇒ Partitioning challenges

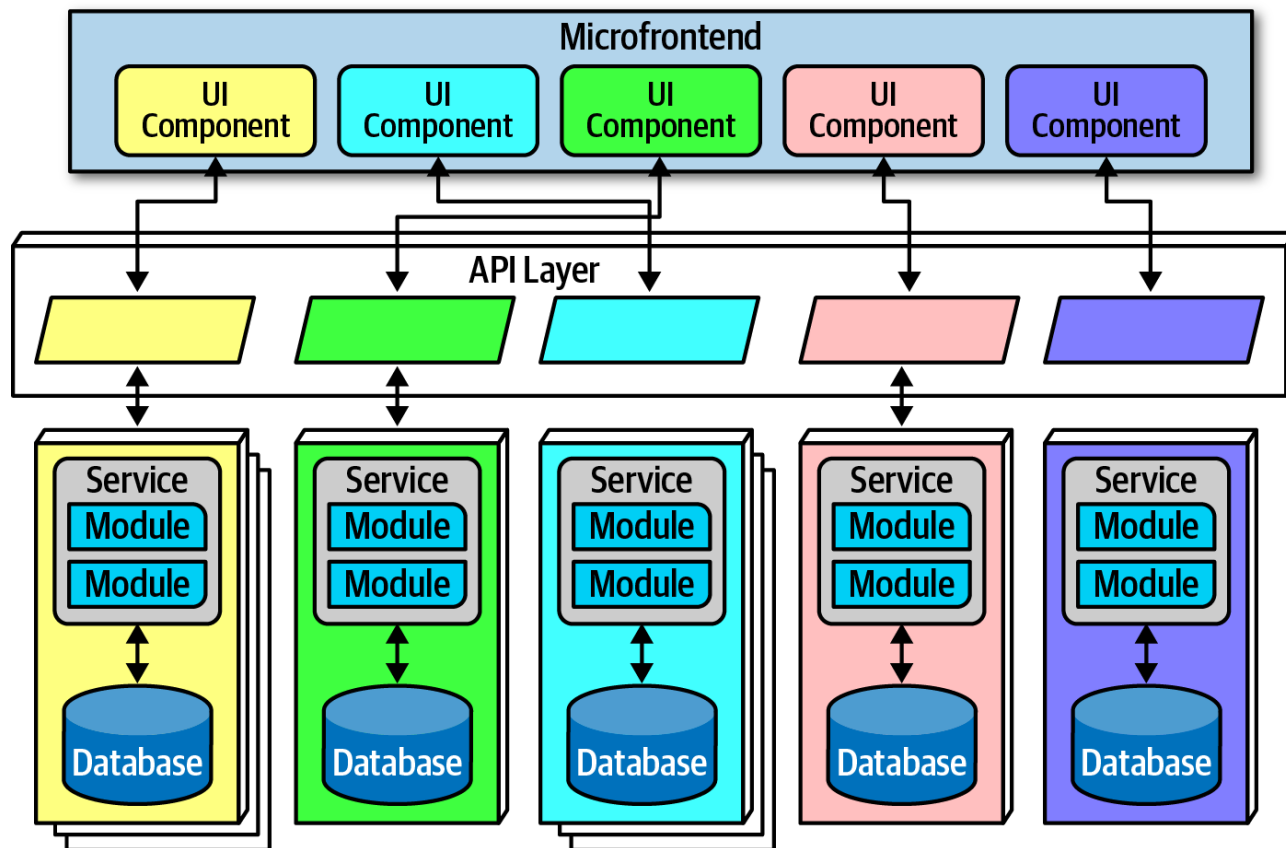
⇒ 2 styles of user interfaces:

Monolithic (ex. rich desktop, mobile, or web application)



MICROFRONTENDS

- Synchronous level of granularity and isolation in the user interface as the backend services



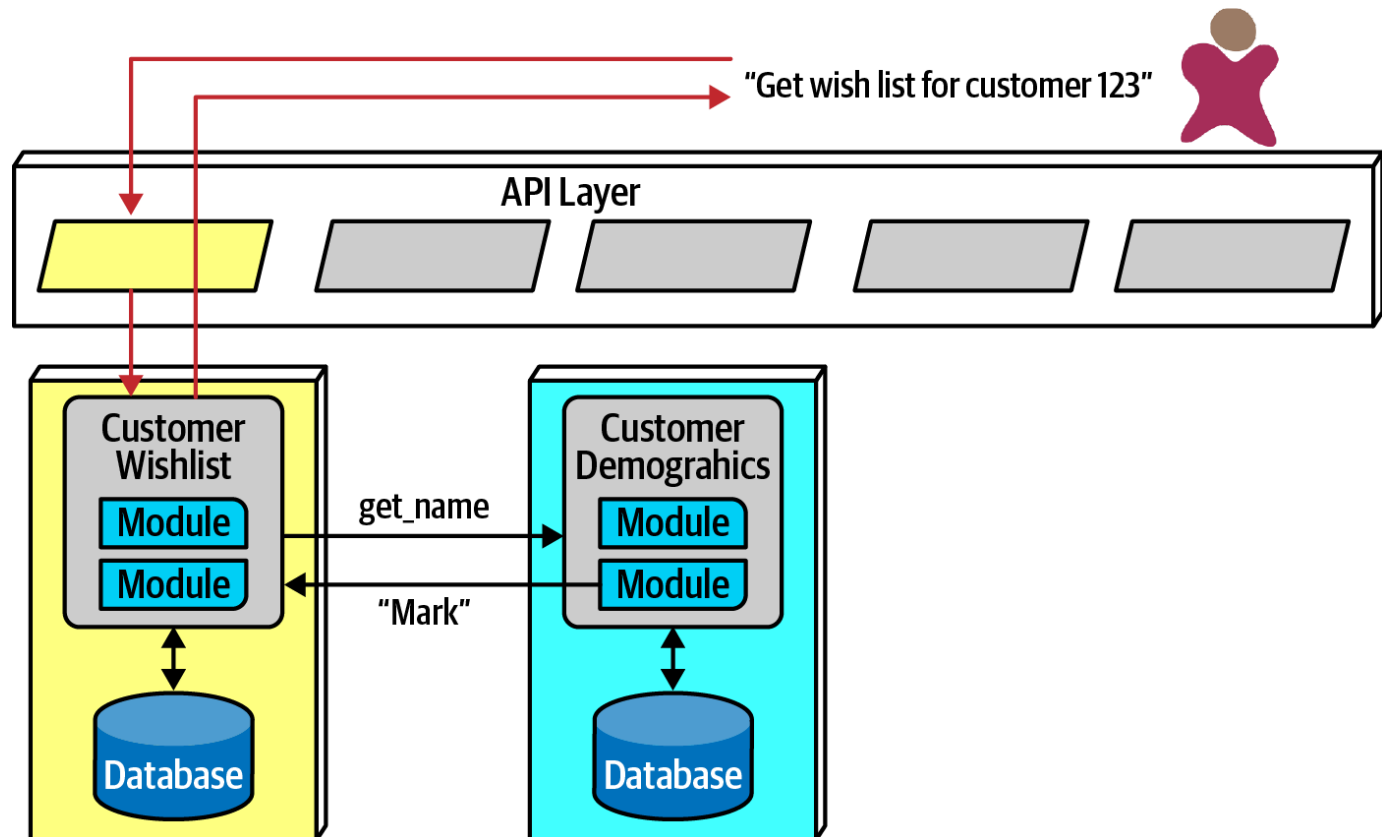
COMMUNICATION

- Synchronous vs asynchronous?
- typically utilize **protocol-aware heterogeneous interoperability**
 - services must know (or discover) which protocol to use to call other services => standardize how services communicate (ex. REST, message queues, etc)
 - each service may be written in a different technology stack => support for polyglot environments
 - services commonly call other services via the network to collaborate and send/receive information
- For asynchronous communication, architects often use events and messages => an event-driven architecture => choreography and orchestration

CHOREOGRAPHY

- Choreography: similar to broker architecture (no central coordinator)
- Each service calls other services as needed, without a central mediator

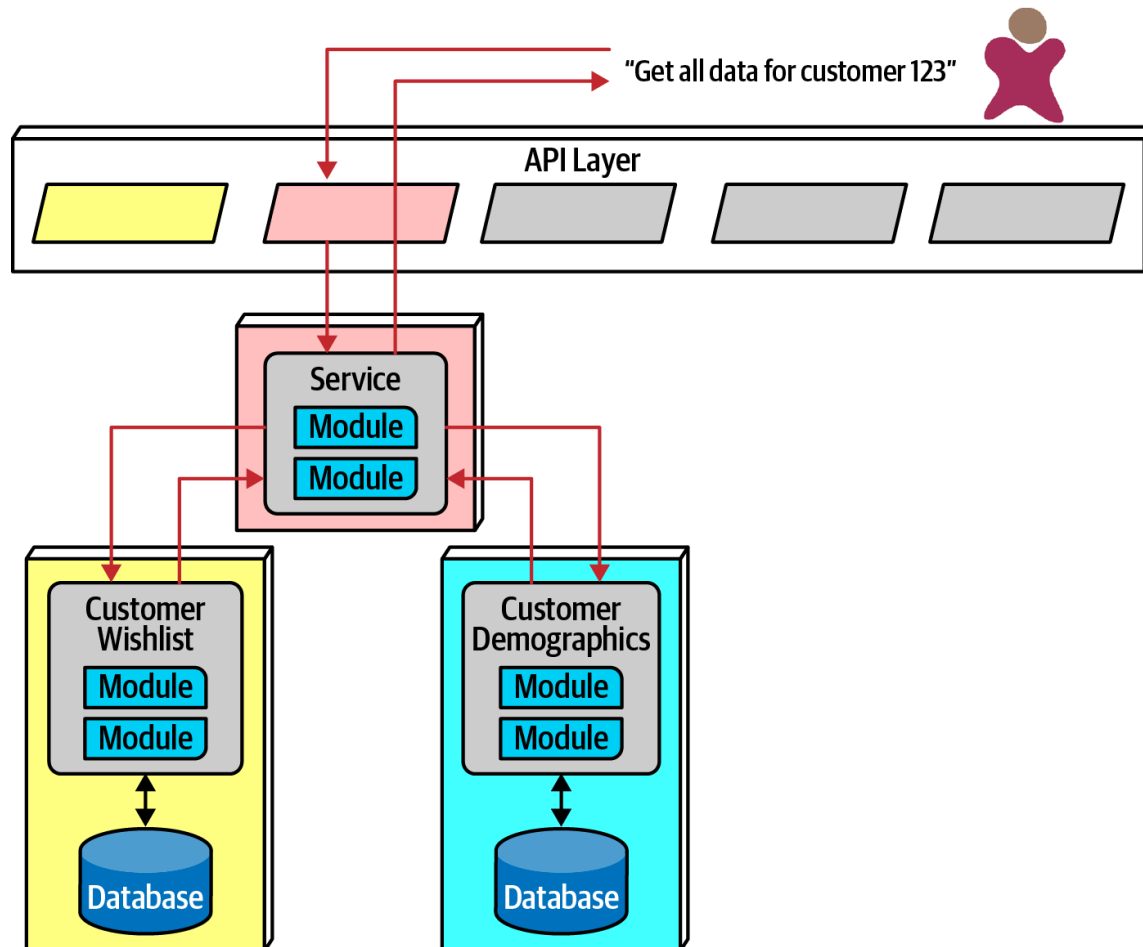
Example



ORCHESTRATION

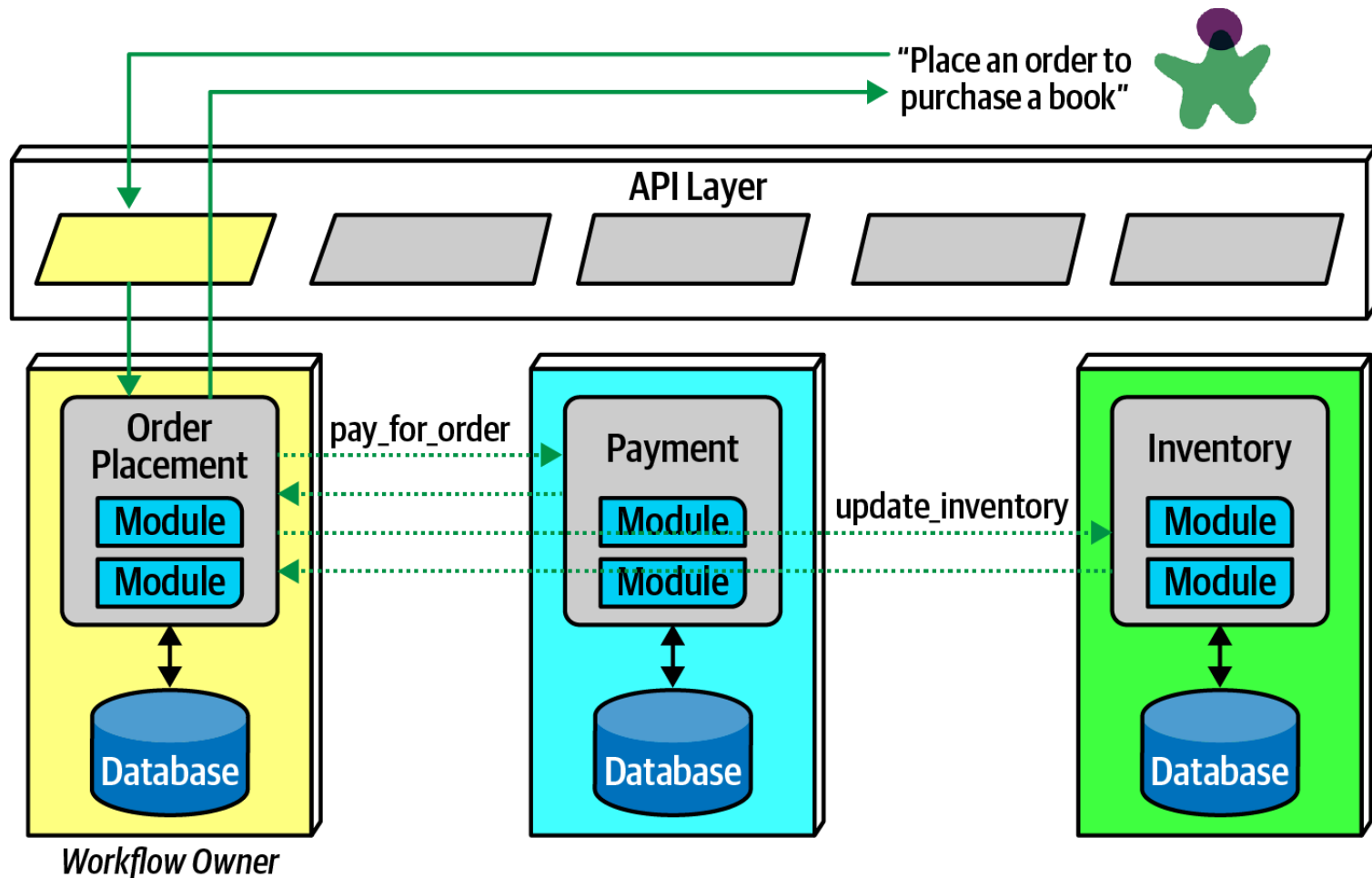
- If coordination across services is needed => local mediator

Example



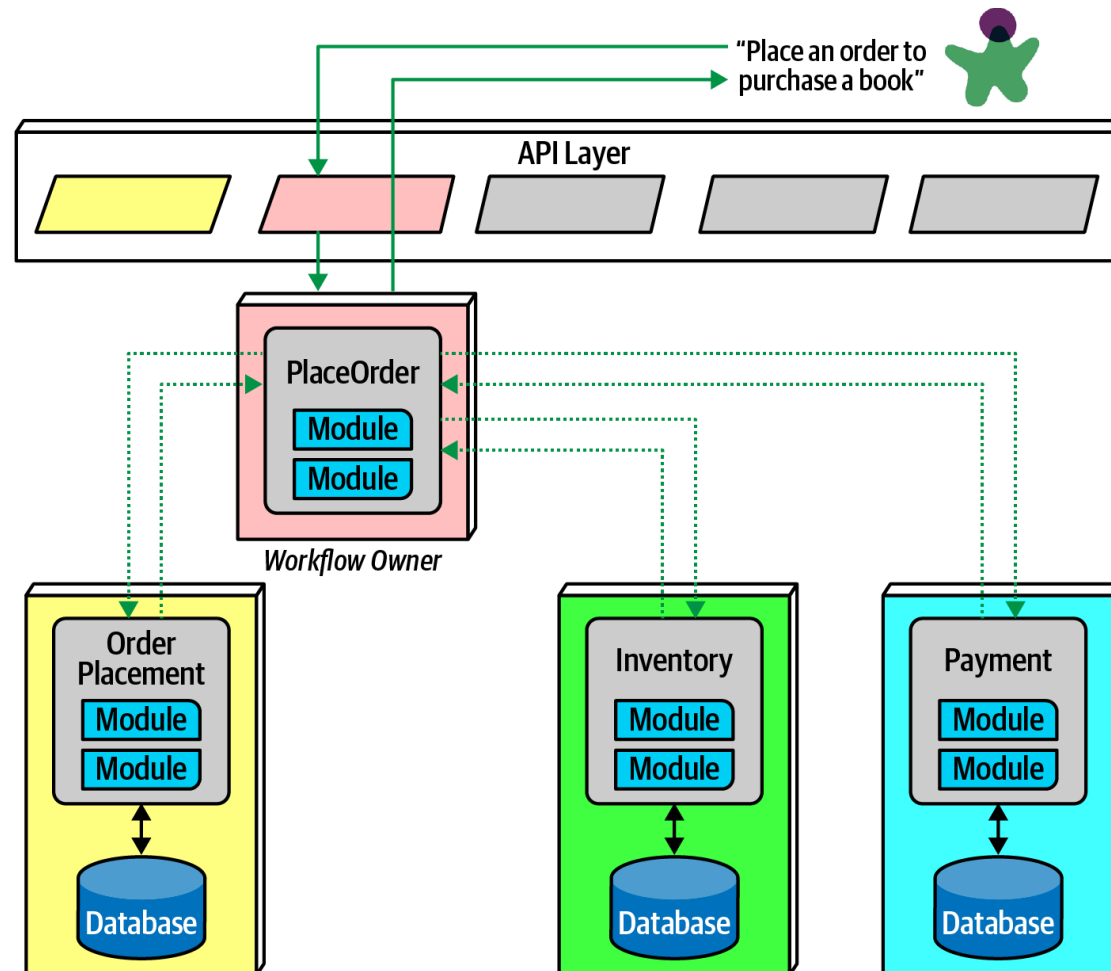
CHOREOGRAPHY — MORE COMPLEX PROCESSES

- First service acts also as mediator (i.e. front controller pattern)



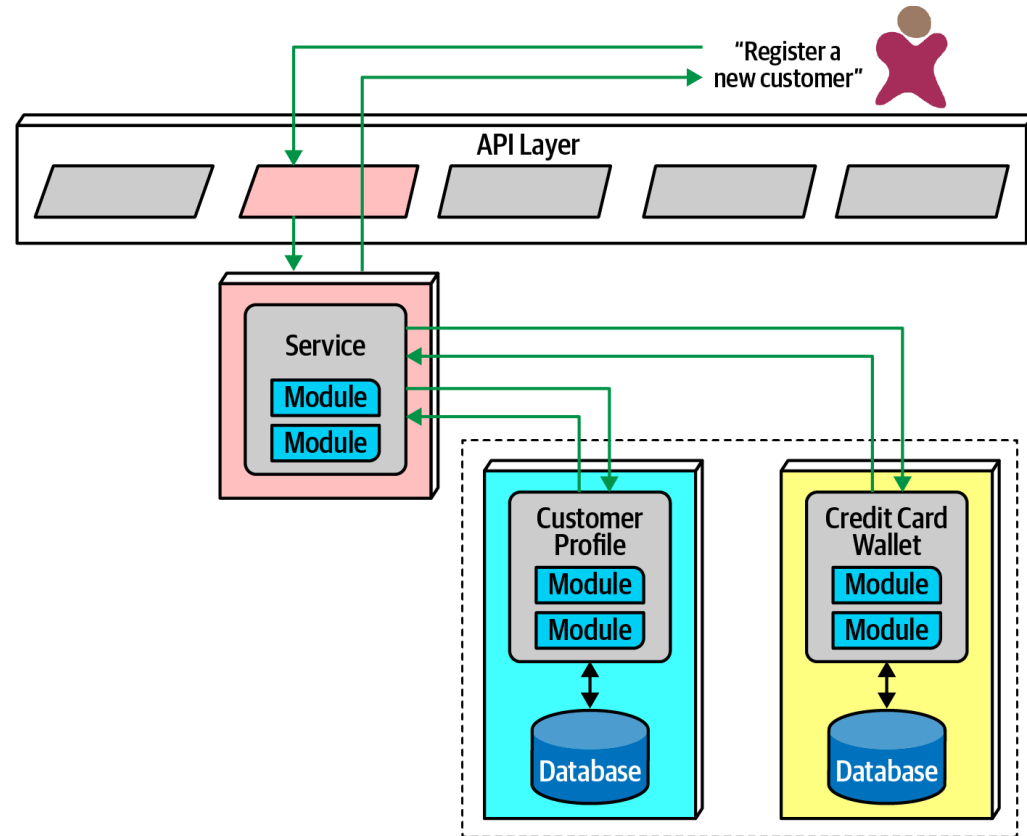
ORCHESTRATION — MORE COMPLEX PROCESSES

- Dedicated mediator

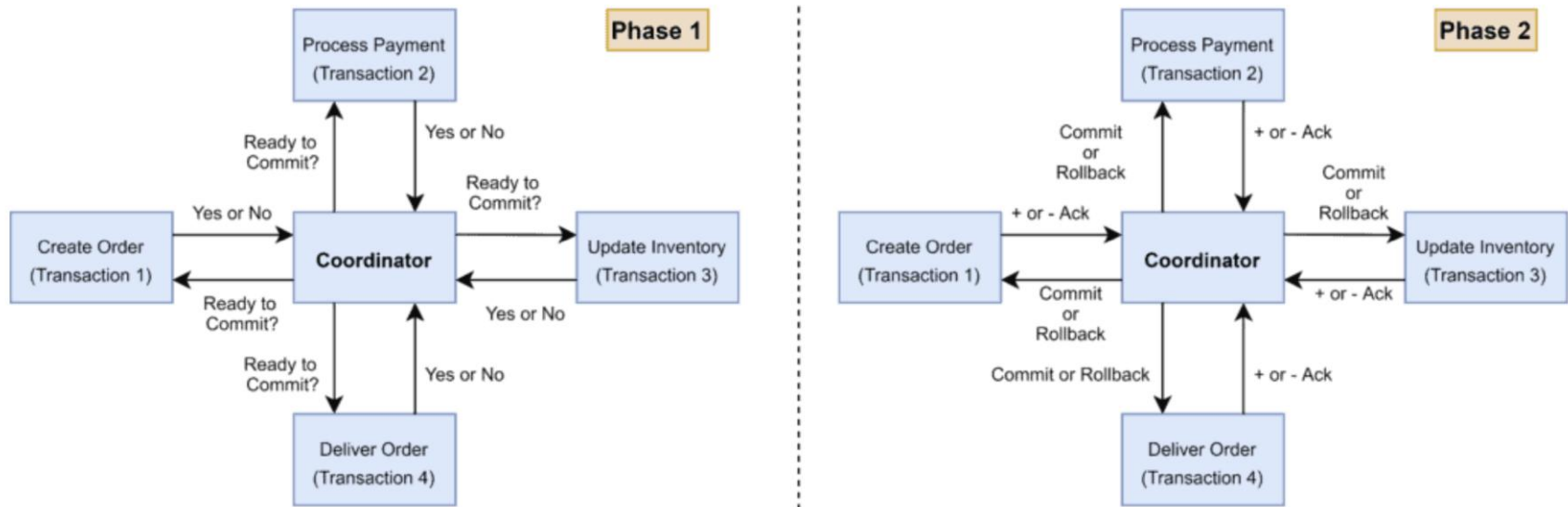


TRANSACTIONS

- Microservices provide extreme decoupling
- ⇒ How to deal with transactions that span several services ?
- ⇒ Advice:
1. DO NOT design transactions across services!
 2. Change granularity of services instead!
 3. If transactional coordination is needed, use patterns



2 PHASE COMMIT PATTERN



- Coordinator = single point of failure
- Overall latency = latency of the slowest service
- Not supported in NoSQL databases

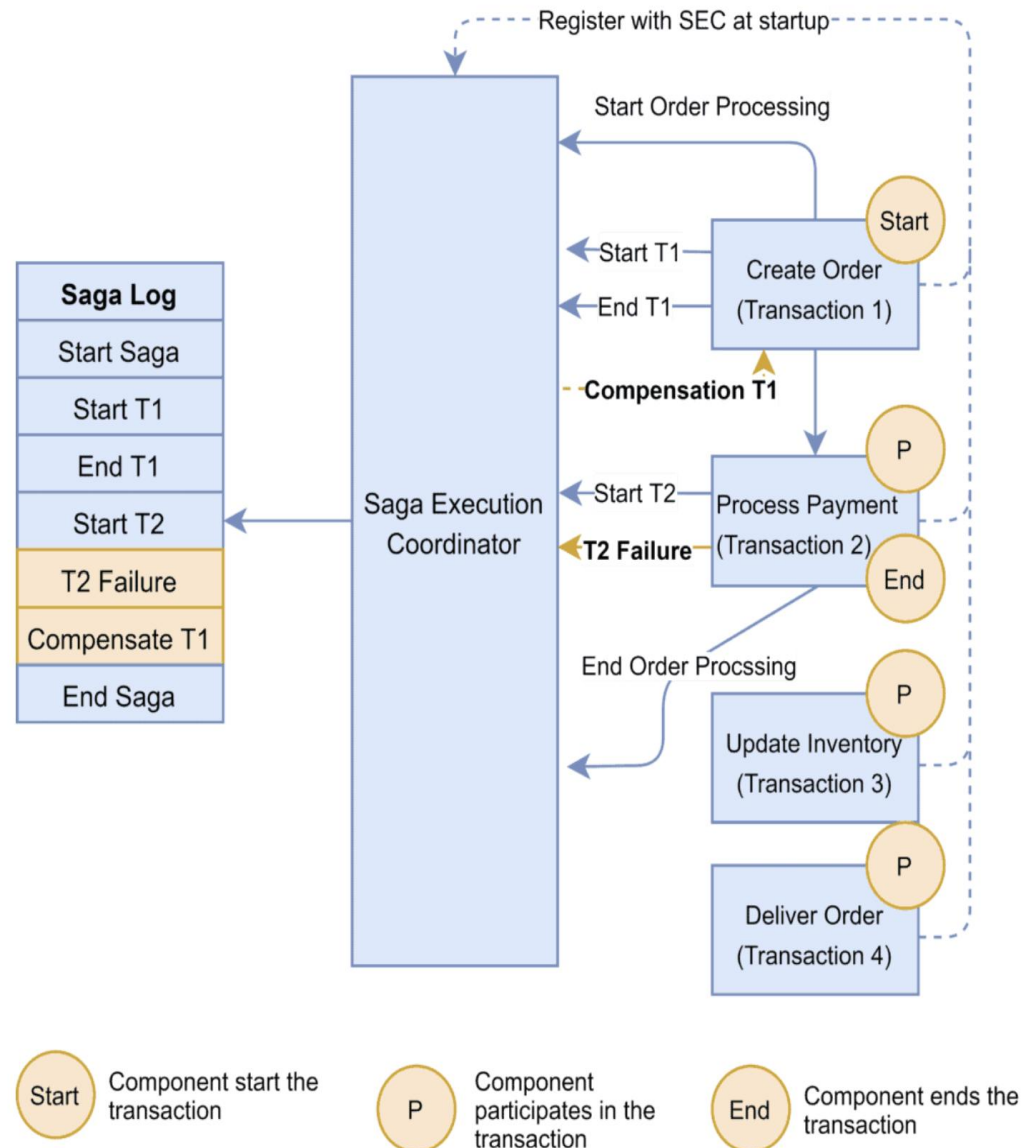
SAGA PATTERN

- A sequence of local transactions
- If an error occurs => the mediator needs to rollback all the operations so far

⇒ compensating transaction framework

⇒ High complexity, high coordination traffic

⇒ **Use the saga pattern sparingly and only if necessary!**



ARCHITECTURE CHARACTERISTICS RATINGS

Architecture characteristic	Star rating
Partitioning type	Domain
Number of quanta	1 to many
Deployability	★★★★
Elasticity	★★★★★
Evolutionary	★★★★★
Fault tolerance	★★★★
Modularity	★★★★★
Overall cost	★
Performance	★★
Reliability	★★★★
Scalability	★★★★★
Simplicity	★
Testability	★★★★

WRAP-UP

- Services should respect a set of principles
- May use different communication means:
 - SOAP
 - REST
- Service-based architectural styles trade-off **Reuse vs. Coupling**
 - Orchestration-based SOA favors reuse => high coupling
 - Microservices favor decoupling => duplication

LET'S SWITCH TO MOODLE AND TAKE A QUIZ

It takes 4 minutes.